

PLACESYNC: STUDENT PLACEMENT TRACKER

AUTOMATED AWS DEPLOYMENT USING INFRASTRUCTURE AS CODE

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Bethala Sumana, APSSDC

Mr. Juttuka Lovababu, APSSDC

Submitted by

Pattan Mahaboob Subhani, 23BQ1A05H4, VVIT Guntur, (Batch -1)

Papadasu Satish, 23BQ1A05G9, VVIT Guntur, (Batch -1)

Netuluri Sphurthi, 23BQ1A05G0, VVIT Guntur, (Batch -1)

Gangavarapu Tharun Kumar, 23BQ1A1235, VVIT Guntur, (Batch -1)

Ikkurthi Lakshmi Narayana, 23BQ1A1243, VVIT Guntur, (Batch -1)

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who have contributed to the successful completion of our summer internship project at Andhra Pradesh Skill Development Corporation (APSSDC). This opportunity has been an enriching and transformative experience for us, and we are truly thankful for the support, guidance, and encouragement we have received along the way.

We extend our sincere regards to Mrs. Bethala Sumana and Mr. Juttuka Lovababu, our mentors, for providing us with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

We would also like to thank the entire team at Andhra Pradesh Skill Development Corporation (APSSDC) for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback we received from our peers significantly contributed to the development and success of this project.

In conclusion, we are honoured to have been a part of this internship program, and we look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Pattan Mahaboob Subhani
Papadasu Satish
Netuluri Sphurthi
Gangavarapu Tharun Kumar
Ikkurthi Lakshmi Narayana

ABSTRACT

This project presents the implementation of PlaceSync, a student placement tracking system deployed on Amazon Web Services (AWS) using a decoupled, cloud-native architecture. The primary goal is to demonstrate Infrastructure as Code (IaC) practices to provision a scalable, independently deployable frontend and backend without manual console configuration.

PlaceSync separates the application into two services: a static frontend (HTML, CSS, and JavaScript) presenting placement drives and company details to students and an administrative dashboard to coordinators, and a Node.js/Express REST API serving data for placement drives and student applications. The frontend is hosted on Amazon S3, while the backend runs on Amazon EC2. Amazon CloudFront sits in front of both, routing "/" to S3 and "/api/*" to EC2 under a single domain, removing the need for CORS and adding CDN distribution.

The backend supports two persistence modes: a local JSON file for development, requiring no AWS dependency, and Amazon DynamoDB for production. AWS IAM authorizes the EC2 instance to access DynamoDB securely, following the principle of least privilege.

The entire AWS infrastructure — S3, EC2, CloudFront, DynamoDB, and IAM — is provisioned using Terraform rather than manual configuration, keeping the deployment reproducible and easy to recreate. This report documents the requirements, design, architecture, data model, API surface, implementation, testing, security, and cost profile of the system, closing with next steps such as CI/CD automation, monitoring, and alerting.

TABLE OF CONTENTS

1. INTRODUCTION5

2. METHODOLOGY.....7

3. TECHNOLOGY STACK9

4. SYSTEM REQUIREMENTS.....11

5. SYSTEM DESIGN / ARCHITECTURE12

6. DATA MODEL.....16

7. API DESIGN17

8. IMPLEMENTATION18

9. TESTING.....21

10. RESULTS12

11. SECURITY CONSIDERATIONS24

12. COST CONSIDERATIONS24

13. FUTURE SCOPE.....25

14. CONCLUSION26

15. REFERENCES.....27

1. INTRODUCTION

1.1. Background

In the digital era, where agility, automation, and reliable access to information have become essential to institutional operations, effective placement tracking has emerged as a real operational need for academic institutions. This project, titled PlaceSync, harnesses cloud-native, Infrastructure-as-Code deployment on Amazon Web Services (AWS) to deliver a decoupled, scalable, and reproducible solution for tracking placement drives and student applications. It demonstrates how modern cloud services and provisioning tools can replace manual, error-prone deployment processes with version-controlled, repeatable infrastructure.

Career and placement offices at academic institutions typically coordinate between many companies, many drives, and hundreds of students each placement season. Without a shared system of record, this coordination tends to fall back on notices, spreadsheets, and email threads, none of which scale gracefully or give students real-time visibility into their own status.

1.2. Problem Statement

Placement information at many institutions is scattered across notices, emails, and spreadsheets, leaving students without a single source of truth for placement drives, company details, and their own application status. Coordinators, in turn, manage company and drive records manually, which does not scale well as the number of drives grows. PlaceSync addresses both problems with a lightweight web application, while using the project as a vehicle to demonstrate hands-on AWS deployment and DevOps practices — the core objective of this internship.

1.3. Objectives

- Give students a single, always-available place to view placement drives, company details, and their own application status.

- Give placement coordinators a simple interface to add, update, and remove company and drive records.
- Architect the application as two independently deployable services — a static frontend and a REST API backend — rather than a single monolith.
- Provision the entire AWS deployment declaratively through Terraform, so the environment is reviewable, version-controlled, and reproducible.
- Demonstrate secure, least-privilege access between AWS services using IAM.

1.4. Scope of the Project

PlaceSync is architected as two decoupled services. Students use a static frontend to view placement drives and company details, apply to a company, and track their application status. Administrators use a separate dashboard page to manage placement drive and company records. Both pages are served as static files from Amazon S3, while all dynamic operations are handled by a Node.js/Express REST API running on Amazon EC2. Amazon CloudFront unifies the two under a single domain, and Amazon DynamoDB provides production-grade persistent storage, with AWS IAM securing the permissions between services. Authentication, CI/CD automation, and monitoring are treated as future scope, discussed later in this report, so that the project remains focused on the core deployment and infrastructure objectives above.

This report provides a comprehensive overview of the system's requirements, design, architecture, data model, API surface, implementation, testing strategy, and results, highlighting the decision-making process, technical integrations, and benefits realized through a decoupled, IaC-driven approach.

2. METHODOLOGY

The development of PlaceSync followed a structured and iterative methodology to ensure the successful fulfillment of project objectives. The methodology was divided into key phases, including requirements gathering, system design, implementation, testing, deployment, and refinement. Each phase was designed to align with best practices in cloud-native development and Infrastructure as Code. The following subsections outline each phase in detail.

2.1. Requirements Gathering

The project began with an analysis of the functional and technical requirements for a student placement tracking solution. Discussions around common placement-coordination challenges helped define the core objectives listed in Section 1.3. The system was scoped to handle placement-drive browsing, company detail views, student applications, application-status tracking, and admin management of company records, while keeping the AWS footprint intentionally small and IaC-driven.

2.2. Design

Based on the identified requirements, a decoupled, cloud-native architecture was designed. The frontend and backend were separated into independently deployable services, communicating over a REST API. Amazon S3 was chosen for static frontend hosting, Amazon EC2 for the backend API, Amazon DynamoDB for production data storage, and Amazon CloudFront to unify both services under a single domain. IAM roles were planned to ensure secure, least-privilege interactions between the EC2 instance and DynamoDB. Full design detail is presented in Section 5 (System Design / Architecture).

2.3. Implementation

The implementation phase focused on translating the design into a fully functional, decoupled application. The backend was developed as a Node.js/Express REST API exposing endpoints for placement drives and student applications. The frontend was developed as static HTML, CSS, and JavaScript pages — a student portal and an admin

dashboard — that call the backend API. The backend was written to support two persistence modes, selected by configuration: a local JSON file for development, and Amazon DynamoDB for production.

2.4. Database Management

A NoSQL approach was adopted for production data storage using Amazon DynamoDB, chosen for its fast, scalable, key-value access pattern and its serverless operational model. The local development mode uses a plain JSON file as a mock database, allowing the application to be run and tested without any AWS dependency. This dual-persistence design keeps local iteration fast while still exercising the real production data path when deployed. A representative schema for both tables is presented in Section 6 (Data Model).

2.5. Testing

The application was tested locally before deployment. Both services were started together using a single root-level command, with the frontend served on port 5000 and the backend API on port 3000. Local testing confirmed the placement-drive and application-tracking flows worked correctly against the local JSON persistence layer. Additional testing detail and a post-deployment verification checklist are presented in Section 9 (Testing).

2.6. Deployment

The system was deployed entirely through Terraform. After configuring the AWS CLI with valid credentials, the Terraform configuration was initialized, previewed, and applied, provisioning Amazon S3, Amazon EC2, Amazon CloudFront, Amazon DynamoDB, and the required AWS IAM roles in a single, repeatable workflow. The frontend build was then synchronized to the newly created S3 bucket using the AWS CLI, and the application was accessed through the CloudFront distribution domain returned in the Terraform outputs.

2.7. Iterative Refinement

Throughout the development cycle, iterative feedback loops were used to refine the system. Adjustments were made to the Terraform variable definitions, IAM policy scopes, and the CloudFront routing rules based on testing results and mentor feedback. The architecture was reviewed periodically to ensure it remained aligned with Infrastructure-as-Code and least-privilege best practices, and to keep it ready for future enhancements such as CI/CD automation.

The methodology adopted for the development of PlaceSync ensured a cloud-native, decoupled, and reproducible solution. By following an Infrastructure-as-Code approach throughout, the project demonstrates the practical application of modern DevOps practices to a real placement-tracking use case.

3. TECHNOLOGY STACK

PlaceSync was developed using a modern, cloud-native technology stack that supports a decoupled architecture, Infrastructure as Code, and dual-mode data persistence. The following technologies and services were used throughout the implementation:

3.1. Cloud Platform

- Amazon Web Services (AWS): The entire project is hosted on AWS, leveraging managed services for storage, compute, database, content delivery, and access control.

3.2. AWS Services

- Amazon S3: Hosts the static frontend web client (student portal and admin dashboard).
- Amazon EC2: Hosts the Node.js/Express backend REST API service.
- Amazon DynamoDB: NoSQL database providing key-value persistent storage for placement drives and student applications.

- Amazon CloudFront: Routes "/" to S3 and "/api/*" to EC2 under one domain, removing the need for CORS, with CDN distribution.
- AWS IAM (Identity and Access Management): Authorizes the EC2 instance to securely access DynamoDB tables.

3.3. Programming & Scripting

- Node.js + Express: Used for writing the backend REST API.
- HTML, CSS, JavaScript: Used for the decoupled static frontend client.
- Terraform (HCL): Used to declaratively define and provision all AWS infrastructure.

3.4. Data Format

- JSON: Used both as the API payload format and as the local-development persistence format (a mock JSON file database).

3.5. Security & Access

- IAM Roles and Policies: Configured to allow only the necessary permissions between the EC2 instance and DynamoDB, following the principle of least privilege.

3.6. Local Orchestration & Tooling

- npm workspaces: Manages dependencies for the root, frontend, and backend packages from a single install command.
- concurrently: Runs the frontend and backend development servers together with one command.
- AWS CLI: Used for authenticating deployment operations and syncing the frontend build to S3.

3.7. Why This Stack

Node.js and Express were chosen for the backend because of their lightweight footprint and wide familiarity, making the API straightforward to reason about and extend. A plain

static frontend (no framework build step) was chosen deliberately to keep the deployment surface simple — the frontend is just files to sync to S3, with no build pipeline to maintain. DynamoDB was chosen over a relational database because the access patterns (look up drives, look up a student's applications) are simple key-based lookups that map naturally onto a NoSQL key-value model, and because it requires no server or patching to operate. Terraform was chosen over manual console configuration because it is cloud-agnostic, widely used in industry, and produces a reviewable, diffable record of the infrastructure alongside the application code.

4. SYSTEM REQUIREMENTS

4.1. Functional Requirements

- Students shall be able to view a list of active placement drives and their company details.
- Students shall be able to submit an application to a company's placement drive.
- Students shall be able to check the status of their submitted applications.
- Administrators shall be able to add a new company/drive record.
- Administrators shall be able to edit an existing company/drive record.
- Administrators shall be able to remove a company/drive record.
- The system shall persist all records so that data is not lost between sessions.

4.2. Non-Functional Requirements

- Availability: The frontend and backend shall be independently deployable, so a backend update does not require redeploying the frontend and vice versa.
- Scalability: The data layer shall handle a growing number of drives and applications without requiring a schema migration or manual resharding.
- Security: Access between AWS services shall follow the principle of least privilege, with no long-lived credentials embedded in application code.

- **Reproducibility:** The entire AWS environment shall be provisionable from a clean account using a documented, repeatable set of commands.
- **Portability:** The backend shall run identically in local development (no AWS dependency) and in production (AWS-backed), to keep the development loop fast.

4.3. Hardware & Software Requirements

Requirement	Detail
Local development machine	Any machine capable of running Node.js 18 or later
Node.js	Version 18.0.0 or later
AWS Account	With permissions to create S3, EC2, CloudFront, DynamoDB, and IAM resources
AWS CLI	Configured with valid credentials via "aws configure"
Terraform CLI	Used to provision and tear down the AWS environment
Git	Used for source control and collaboration

5. SYSTEM DESIGN / ARCHITECTURE

PlaceSync is designed as a decoupled, cloud-native solution that separates presentation, delivery, application logic, data storage, and security into distinct layers, each backed by a managed AWS service. The system follows a multi-tier design comprising the presentation layer, the delivery/routing layer, the application layer, the data layer, and the security layer, all provisioned as code.

This section outlines the core components of the architecture and their interactions.

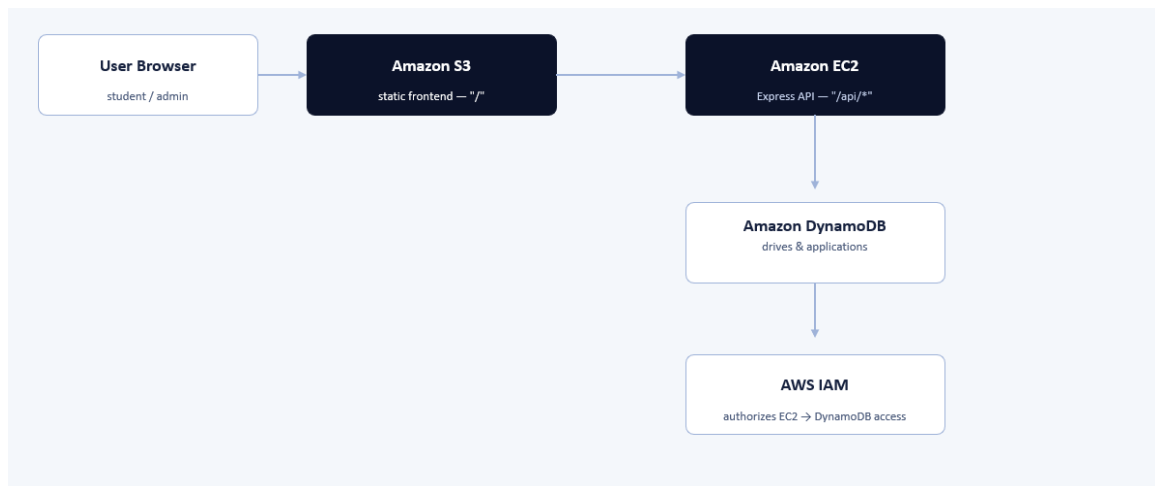


Figure 1. PlaceSync high-level architecture

5.1. Presentation Layer — Amazon S3

The frontend consists of static HTML, CSS, and JavaScript files: a student portal for viewing placement drives and applying to companies, and an admin dashboard for managing company and drive records. These files are hosted directly from an Amazon S3 bucket configured for static website hosting, requiring no server to render pages.

- Serves the student portal and admin dashboard as static files
- Scalable and cost-effective storage with no servers to manage
- Updated by syncing the built frontend directory directly to the bucket

5.2. Application Layer — Amazon EC2

The backend REST API is a Node.js/Express application running on an Amazon EC2 instance. It exposes endpoints for retrieving placement drives and company details, submitting student applications, tracking application status, and managing company records from the admin dashboard. The API reads its persistence mode from configuration, allowing it to run against either the local JSON store or Amazon DynamoDB without code changes.

5.3. Data Storage — Amazon DynamoDB (with local JSON for development)

In production, placement drive and application records are stored in Amazon DynamoDB, a fast, fully managed NoSQL database. In local development, the same backend code reads and writes a JSON file instead, so the application can be built and tested end-to-end without any AWS dependency. This dual-mode design keeps local iteration fast while still exercising the real production data path once deployed.

5.4. Security and Access Control — AWS IAM

AWS Identity and Access Management (IAM) is used to securely grant permissions between services:

- The EC2 instance is granted a role that permits read and write access to the DynamoDB tables it needs, and nothing more.
- Least-privilege access policies are enforced to reduce the blast radius of any single compromised component.

5.5. Infrastructure as Code — Terraform

All of the AWS resources described above — the S3 bucket, the EC2 instance, the CloudFront distribution, the DynamoDB tables, and the IAM roles — are declared in Terraform configuration files rather than created by hand in the AWS console.

- `main.tf`: Declares and provisions the S3 bucket, EC2 instance, CloudFront distribution, DynamoDB tables, and IAM roles.
- `variables.tf`: Exposes configurable parameters such as AWS region and EC2 instance type.
- `outputs.tf`: Surfaces the resulting CloudFront distribution domain and backend endpoint after provisioning.

Provisioning the environment through Terraform makes the infrastructure reviewable, version-controlled alongside the application code, and reproducible — the same configuration can be applied again to recreate an identical environment in a new AWS account or region.

5.6. Repository Structure

The project repository separates application code from infrastructure code:

Path	Purpose
backend/	Node.js/Express REST API source code
backend/src/	Application code (routes, handlers, persistence logic)
backend/.env.example	Backend environment variable template
frontend/	Decoupled static HTML, CSS, and JS web client
frontend/index.html	Student portal
frontend/admin.html	Admin dashboard
terraform/main.tf	Provisions S3, EC2, CloudFront, DynamoDB, and IAM
terraform/variables.tf	Configurable region and instance parameters
terraform/outputs.tf	CloudFront and backend access endpoints
package.json	Root orchestration for running both services locally

5.7. Request Lifecycle

A typical student-facing request flows through the architecture as follows:

- 1. The student's browser requests the CloudFront distribution domain.
- 2. For a page load ("/"), CloudFront serves the static frontend from the S3 origin.
- 3. The loaded frontend JavaScript calls the API ("/api/..."), which CloudFront routes to the EC2 origin instead.
- 4. The Express application on EC2 handles the request, reading from or writing to DynamoDB (or the local JSON store in development).
- 5. The response travels back through CloudFront to the browser, completing the request-response cycle without any CORS negotiation, since both origins are presented under the same domain.

6. DATA MODEL

The tables below present a representative schema for the two core record types PlaceSync manages. Field names should be confirmed against the team's actual Terraform table definitions and backend persistence code before being treated as final.

6.1. Placement Drives / Companies

Attribute	Type	Description
driveId (Key)	String	Unique identifier for the placement drive / company record
companyName	String	Name of the hiring company
role	String	Role or position being hired for
description	String	Company/role details shown to students
driveDate	String	Scheduled date of the placement drive
status	String	Whether the drive is currently open or closed

6.2. Applications

Attribute	Type	Description
applicationId (Key)	String	Unique identifier for the application record
driveId	String	Reference to the placement drive applied to
studentId	String	Identifier for the applying student
appliedAt	String	Timestamp of submission
status	String	Current status of the application (e.g., submitted, under review)

7. API DESIGN

The backend exposes a REST API consumed by both the student portal and the admin dashboard. The endpoint surface below reflects the application's described functionality; exact route paths should be confirmed against the team's backend/src route definitions.

7.1. Student-Facing Endpoints

Method & Path	Description
GET /api/drives	List all active placement drives
GET /api/drives/:id	Get details for a specific company/drive
POST /api/applications	Submit a student application to a drive
GET /api/applications/:studentId	Get application status for a student

7.2. Admin-Facing Endpoints

Method & Path	Description
POST /api/admin/drives	Add a new company/drive record
PUT /api/admin/drives/:id	Update an existing company/drive record
DELETE /api/admin/drives/:id	Remove a company/drive record

8. IMPLEMENTATION

The implementation of PlaceSync translates the architectural design into a fully functional, decoupled application. This section outlines how each AWS service and application component was configured and developed to work together.

8.1. Frontend Development and S3 Hosting

The frontend was built as static HTML, CSS, and JavaScript, split into a student portal (index.html) and an admin dashboard (admin.html). Both pages call the backend REST API for all dynamic data. The built frontend directory is synchronized to an Amazon S3 bucket configured for static website hosting using the AWS CLI.

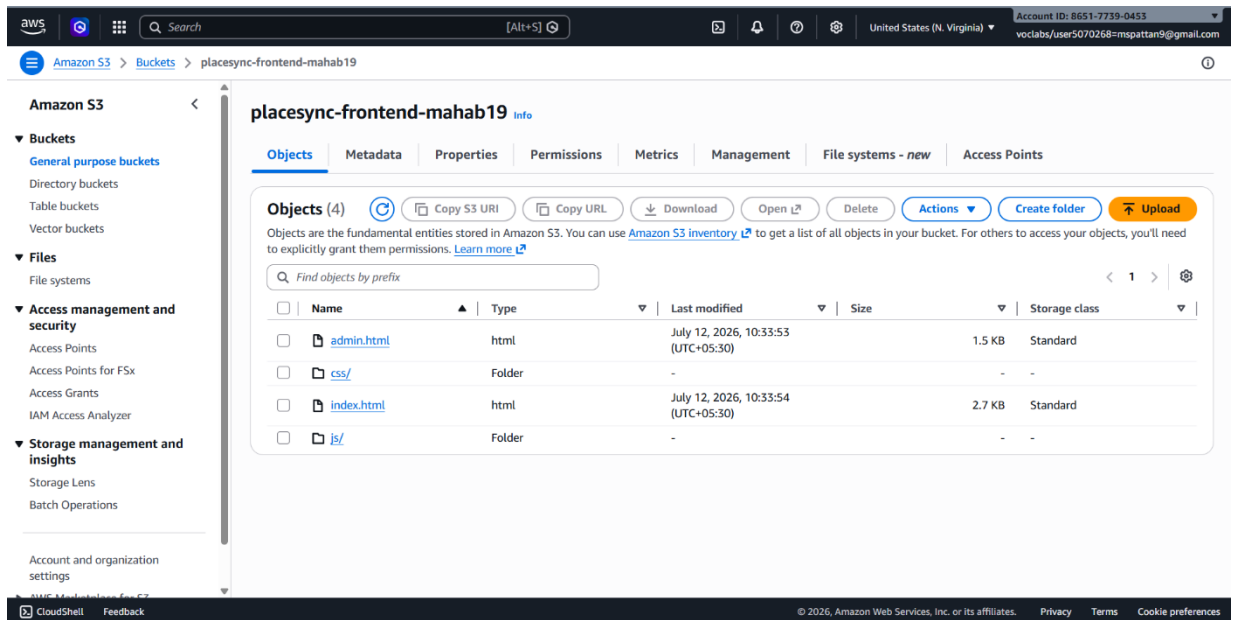


Figure 2. S3 static website hosting configuration

8.2. Backend API Development — Node.js and Express

The backend REST API, running on Amazon EC2, performs the following tasks:

- Exposes endpoints to list placement drives and company details
- Accepts student applications and records their initial status
- Returns a student's application status on request
- Exposes admin endpoints to add, update, and remove company/drive records
- Reads its persistence mode (local JSON or DynamoDB) from environment configuration

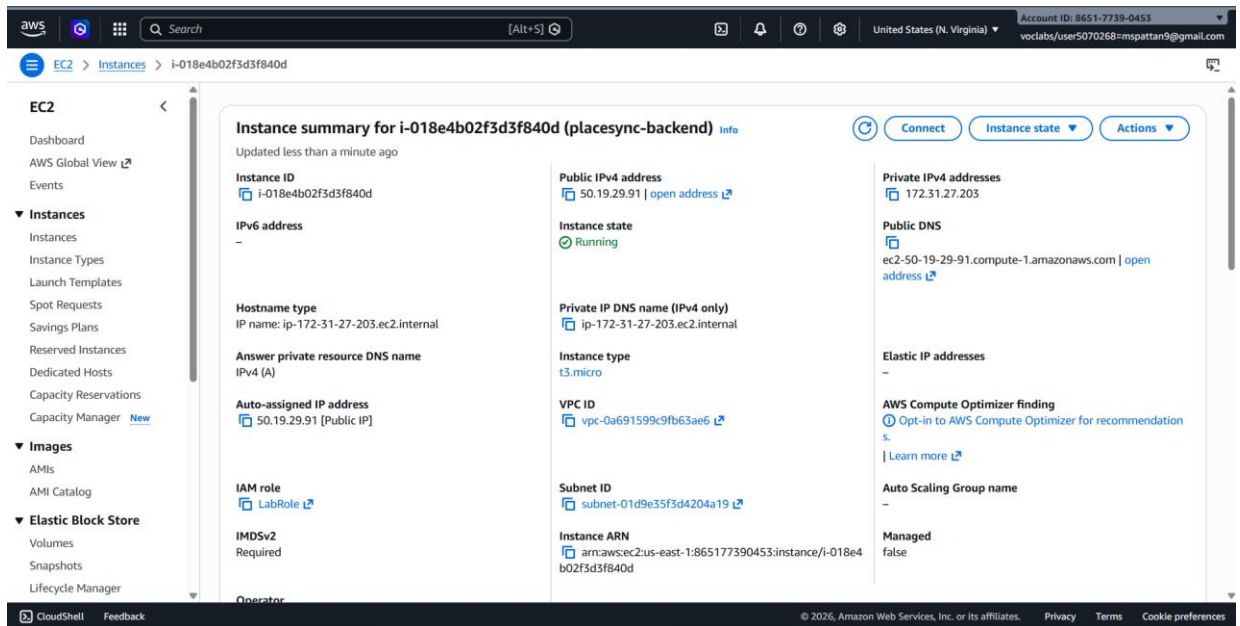


Figure 3. EC2 instance hosting the backend API

8.3. Data Storage Implementation

A DynamoDB table is provisioned to store placement drive and application records for production use. The local development mode uses a JSON file at a fixed path within the backend project as a stand-in database, so the same codebase can run identically with or without AWS.

8.4. CloudFront Distribution Configuration

A CloudFront distribution is configured with two origins: the S3 bucket for the default "/" behavior, and the EC2 instance for the "/api/*" path pattern. This single distribution becomes the one public entry point for the entire application, eliminating the need for CORS headers between the frontend and backend.

8.5. IAM Roles and Security

All interactions between services are secured using IAM roles with the principle of least privilege:

- EC2 instance role: Access to read and write the DynamoDB tables used by the application, and nothing else.

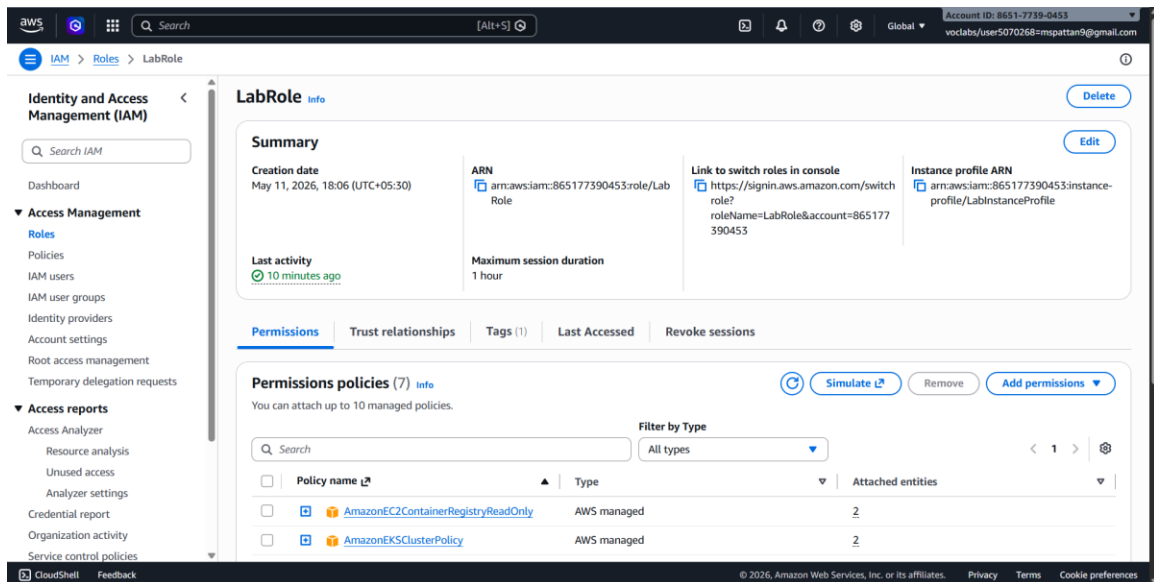


Figure 4. IAM role authorizing EC2 access to DynamoDB

8.6. Infrastructure Provisioning via Terraform

The AWS environment is provisioned with a small, repeatable sequence of commands:

- `aws configure` — sets up AWS CLI credentials
- `terraform init` — initializes the Terraform working directory
- `terraform plan` — previews the infrastructure changes to be made
- `terraform apply` — provisions S3, EC2, CloudFront, DynamoDB, and IAM
- `aws s3 sync` — uploads the built frontend to the new S3 bucket

The successful implementation of PlaceSync shows how a decoupled application can be built and provisioned entirely through code, with each AWS service handling a single, well-defined responsibility.

9. TESTING

9.1. Local Testing Strategy

Before any AWS deployment, the application was run and tested entirely locally. Running the root command starts the frontend on port 5000 and the backend API on port 3000 together, with the backend defaulting to its local JSON persistence mode. This allowed the student-facing flows (browsing drives, applying, tracking status) and the admin flows (adding, editing, and removing drives) to be exercised end-to-end without incurring any AWS cost or requiring AWS credentials.

9.2. Deployment Verification Checklist

After provisioning the AWS environment with Terraform, the following checks confirm the deployment is healthy:

- The CloudFront distribution domain from the Terraform outputs loads the student portal at "/".
- The admin dashboard loads correctly at its expected path.
- A request to an "/api/*" path returns data from the backend, confirming CloudFront routing to EC2 is correct.
- Submitting a test application through the frontend results in a new record visible through the tracking flow.
- Adding a company through the admin dashboard results in a new drive visible in the student portal.
- The EC2 instance's IAM role permits the expected DynamoDB operations and nothing beyond them.

10. RESULTS

The implementation of PlaceSync was validated against the project's core objectives: a decoupled frontend and backend, dual-mode data persistence, and infrastructure that can be provisioned and torn down entirely through Terraform. The subsections below describe the expected and observed outcomes; the team should replace the bracketed notes with their own testing evidence and screenshots before final submission.

10.1. Decoupled Frontend and Backend

With CloudFront routing "/" to S3 and "/api/*" to EC2 under a single domain, the frontend and backend operate as independent, separately deployable services while presenting as one application to the end user, with no CORS configuration required in either service.

10.2. Infrastructure Reproducibility

Because every AWS resource is defined in Terraform, the full environment — S3, EC2, CloudFront, DynamoDB, and IAM — can be provisioned from a clean AWS account with a single terraform apply, and torn down completely with terraform destroy, without any manual console steps.

10.3. Dual-Mode Persistence

The backend runs identically against the local JSON file in development and against DynamoDB in production, allowing the team to iterate locally without incurring any AWS cost or requiring AWS credentials during day-to-day development.

10.4. Secure, Least-Privilege Access

The EC2 instance is only able to access the specific DynamoDB tables it needs, confirmed through the IAM role and policy attached to the instance, rather than being granted broad account-level permissions.

10.5. Screenshots of Output

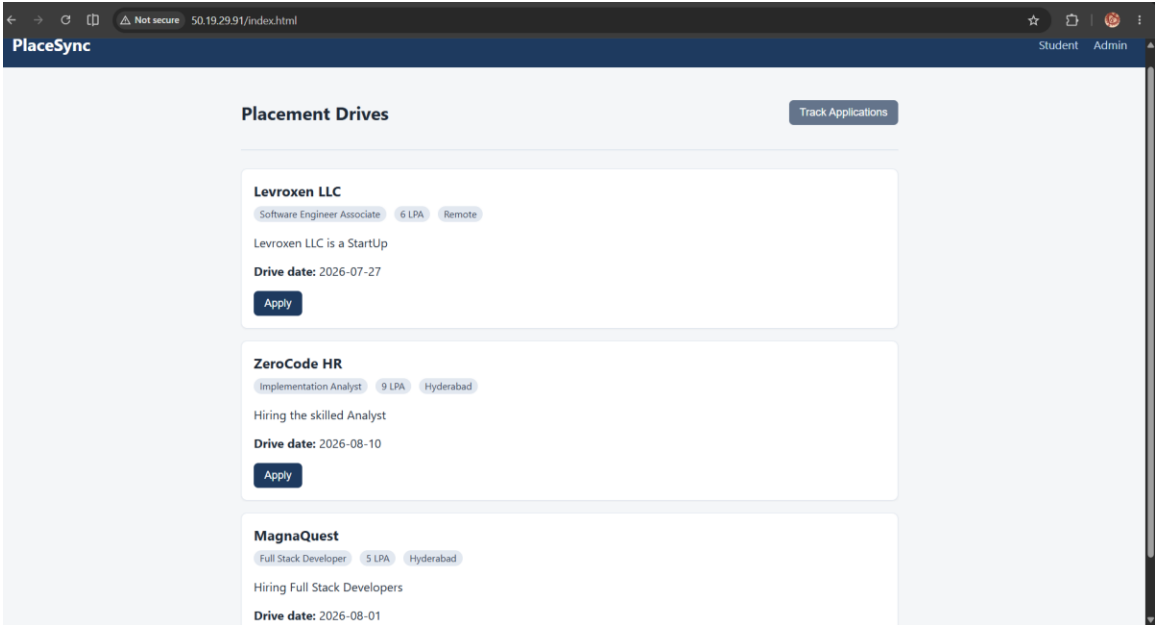
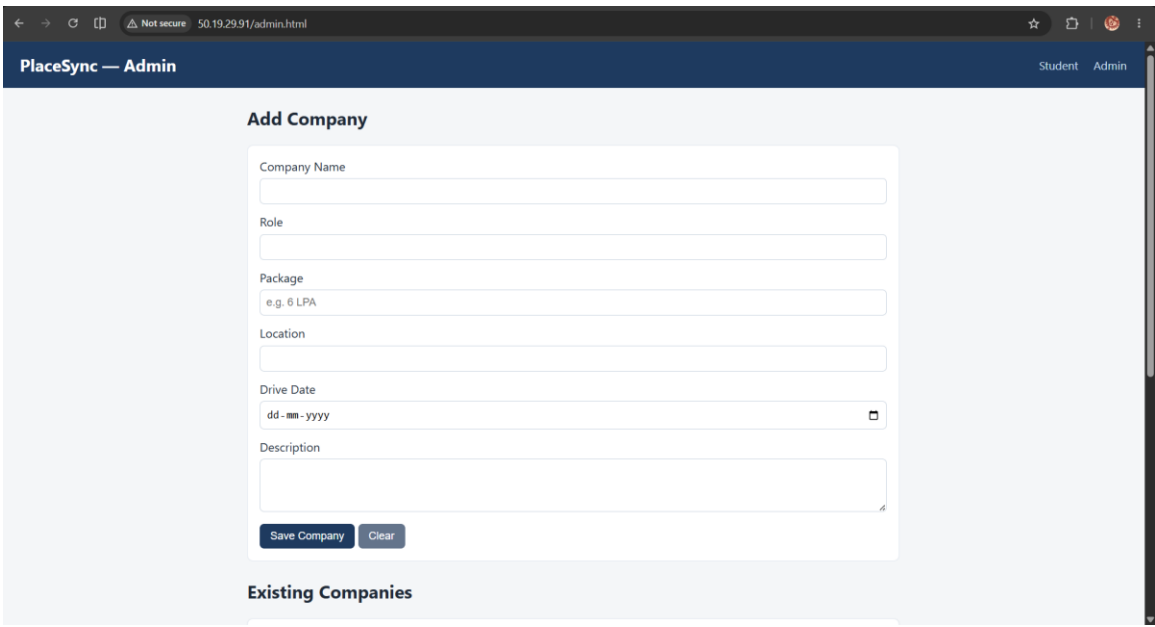


Figure 5. Student portal



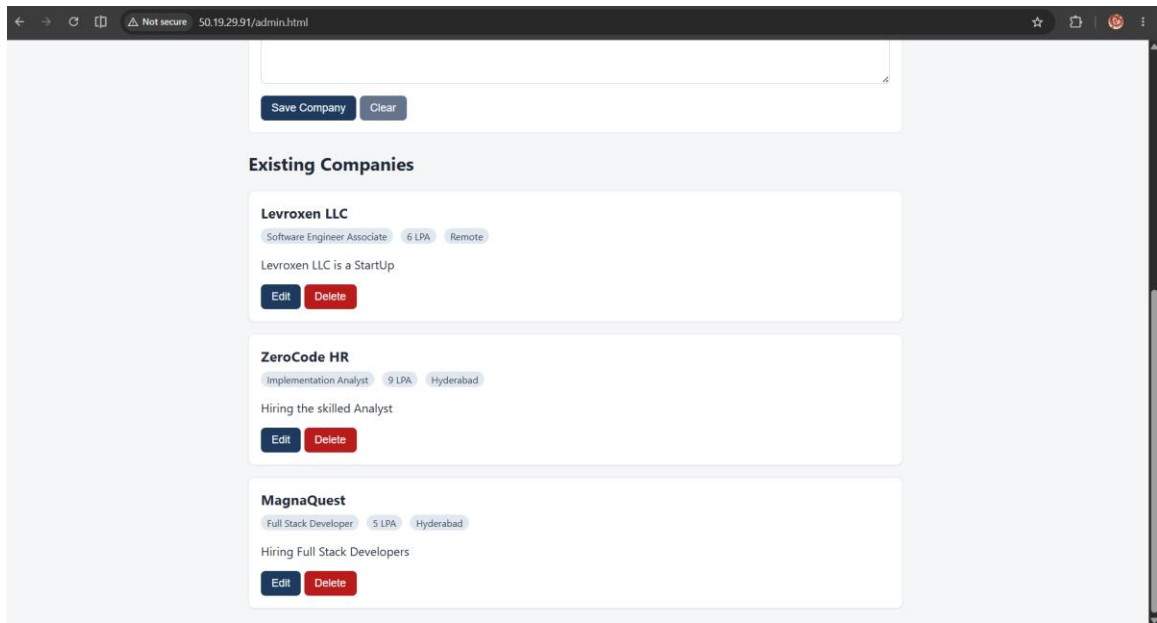


Figure 6. Admin dashboard

The overall results demonstrate that the decoupled, IaC-provisioned architecture meets its intended goals of separation of concerns, reproducibility, and secure access. The architecture is stable and ready for further extension, including CI/CD automation and monitoring.

11. SECURITY CONSIDERATIONS

- Least privilege: The EC2 instance role grants only the DynamoDB actions the application actually needs, not broad table or account-level access.
- No embedded credentials: The backend authenticates to DynamoDB through its attached IAM role rather than access keys stored in code or configuration files.
- Single entry point: Routing both the frontend and backend through one CloudFront distribution reduces the exposed surface area compared to publishing the EC2 instance directly.

- Infrastructure review: Because all resources are declared in Terraform, IAM policy changes are reviewable as part of normal code review, rather than being made ad hoc in the console.
- Known limitation: The current implementation does not include application-level authentication for students or administrators; this is addressed in Section 13 (Future Scope).

12. COST CONSIDERATIONS

PlaceSync was designed to be inexpensive to run for a project of this scale. The dominant cost drivers are the EC2 instance (billed hourly while running), CloudFront (billed per request and data transfer), DynamoDB (billed per request under on-demand capacity, or a small fixed cost under provisioned capacity), and S3 (billed per GB stored and per request, both negligible for a static frontend of this size).

- EC2: Choose the smallest instance type that comfortably runs the Node.js API; stop or terminate the instance via Terraform when not actively in use.
- DynamoDB: On-demand capacity mode avoids paying for idle provisioned throughput during development and demos.
- S3 and CloudFront: Costs scale with actual student/admin traffic and are minimal at internship-project scale.
- Because the entire environment is defined in Terraform, it can be destroyed with "terraform destroy" between working sessions to avoid any idle cost, and recreated identically when needed.

13. FUTURE SCOPE

- Automate the provisioning and deployment pipeline with GitHub Actions CI/CD.
- Add Amazon CloudWatch dashboards and alarms for monitoring the EC2 backend.
- Integrate Amazon SNS for deployment and alarm notifications.

- Add authentication for student and admin roles.
- Attach a custom domain with HTTPS via an ACM certificate on CloudFront.
- Introduce automated tests (unit and integration) run as part of the deployment pipeline.
- Add pagination and search to the placement-drive listing as the number of drives grows.

14. CONCLUSION

The successful development and deployment of PlaceSync demonstrate the practicality of a decoupled, cloud-native architecture provisioned entirely through Infrastructure as Code. By leveraging Amazon S3, Amazon EC2, Amazon CloudFront, Amazon DynamoDB, and AWS IAM — all defined in Terraform — the system achieves its core objectives: giving students a single place to view placement drives and track applications, giving coordinators a way to manage company records, and giving the team a reproducible, version-controlled deployment process.

The project highlights the benefits of a decoupled architecture and Infrastructure as Code, including independent deployability of the frontend and backend, reproducible environments, and a clear separation of concerns between presentation, delivery, application logic, and data storage. The dual-mode persistence design allowed the team to iterate quickly during development without any AWS dependency, while still exercising the real production data path through DynamoDB once deployed.

Throughout the development lifecycle, careful attention was given to security and modularity. The system was designed using least-privilege IAM permissions between the EC2 instance and DynamoDB, and CloudFront was used to unify the frontend and backend under a single domain without relying on CORS. Testing confirmed the application functions correctly against both persistence modes.

In conclusion, this project serves as a practical, reproducible blueprint for decoupled application deployment on AWS, particularly in domains where separation of concerns and infrastructure reproducibility are valuable. It showcases how Infrastructure as Code and managed AWS services can be strategically combined to build a scalable, secure, and future-ready solution.

15. REFERENCES

- AWS Documentation — Amazon S3: <https://docs.aws.amazon.com/s3/>
- AWS Documentation — Amazon EC2: <https://docs.aws.amazon.com/ec2/>
- AWS Documentation — Amazon DynamoDB:
<https://docs.aws.amazon.com/dynamodb/>
- AWS Documentation — Amazon CloudFront:
<https://docs.aws.amazon.com/cloudfront/>
- AWS Documentation — AWS IAM: <https://docs.aws.amazon.com/iam/>
- Terraform Documentation — HashiCorp:
<https://developer.hashicorp.com/terraform>
- Project Repository: github.com/Mahaboob19/APSSDC-Internship-Project